

Apply Interpretability on Generalized Additive Models

- In statistics, a Generalized Additive Model (GAM) is a Generalized Linear Model (GLM) in which the **linear response variable depends linearly on unknown smooth functions of some predictor variables**, and interest focuses on inference about these smooth functions.
- **GAM** is a statistical or machine learning model that is used for **regression** and **classification** problems, especially when the relationship between predictors (input features) and the response (target) is **non-linear** but still **additive**.

GAM

- A Generalized Additive Model has the form:

$$g(E[Y]) = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p)$$

- g : link function (e.g., logit for binary classification)
- $E[Y]$: expected value of the response variable
- β_0 : intercept
- $f_i(X_i)$: smooth, non-linear functions of the predictors

Each f_i is often estimated using **splines** or **kernel smoothers**.

GAM

- Unlike linear models that assume a straight-line relationship between each feature and the target, **GAMs allow for non-linear relationships. However, they still assume the effects of features are additive** (i.e., no interactions by default unless specified).
- Additive because the model adds up the contributions of each feature independently. No feature interacts with another unless explicitly modeled.

Smooth function

- In Generalized Additive Models (GAMs), a smooth function refers to a non-linear function $f(x)$ that estimates how the predictor x affects the response variable without assuming a strict linear form.
- A smooth function is:

Flexible: it can bend to fit the data

Data-driven: its shape is learned from data

Non-parametric or semi-parametric(does not assume a specific functional form like linear, quadratic, etc.)

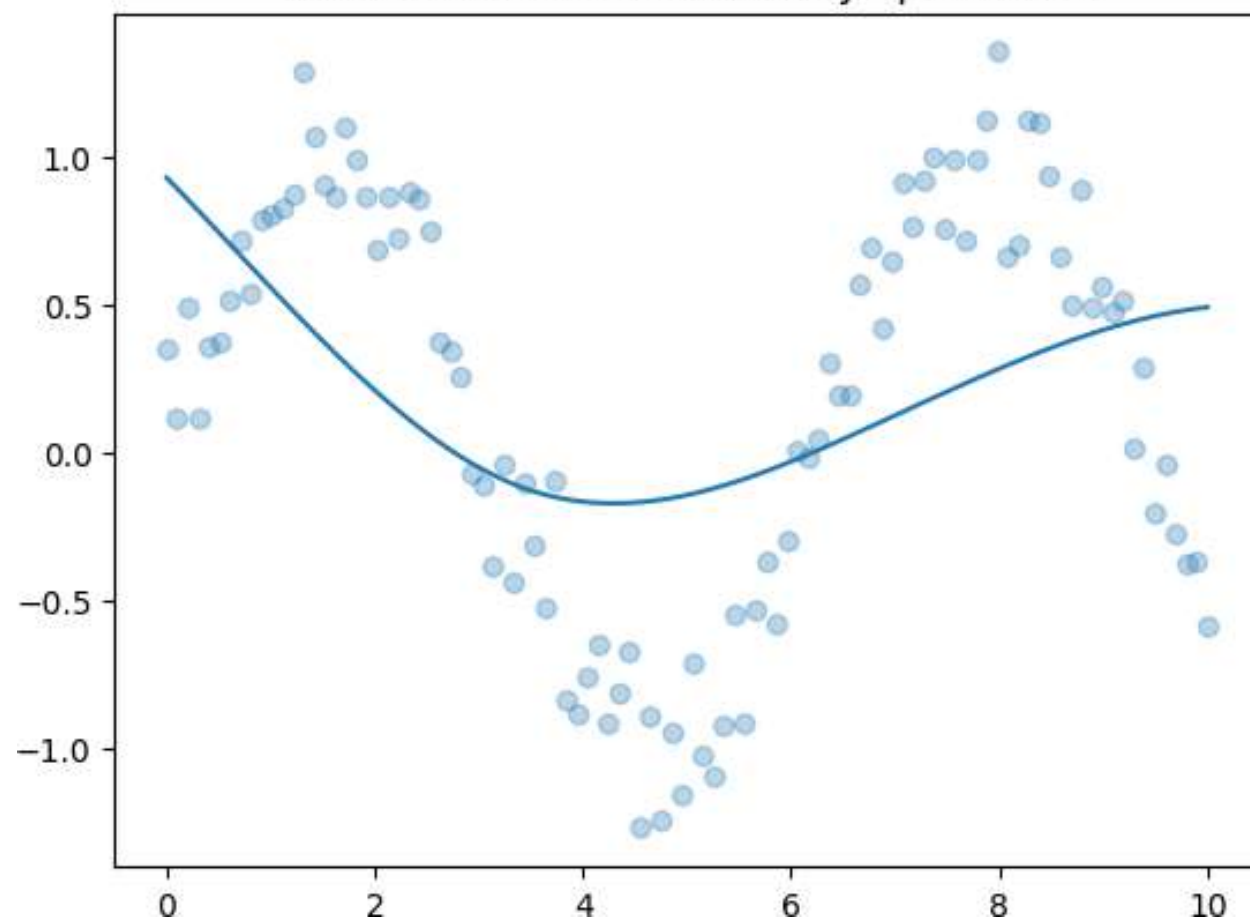
How to learn nonlinear function

“Splines” or “spline functions”.

- Splines are functions that are constructed from simpler basis functions.
- Splines can be used to approximate other, more complex functions.
- Each smooth term $f_i(X_i)$ is modeled as a spline, i.e., a sum of weighted spline basis functions.
- The model automatically learns how to "bend" the function smoothly through the data points.
- This makes GAMs powerful tools for capturing non-linear relationships while keeping the model interpretable.

- Splines are smooth piecewise polynomial functions.
- In GAM, they are used to flexibly model nonlinear relationships between each feature and the response y , without assuming a straight line.

Smooth Function Estimated by Spline Basis



- pip install pygam
- from pygam import LinearGAM, s

#LinearGAM is used to fit a GAM for regression problems.

#s defines a spline term (i.e., a smooth function).

#Fit a GAM with a spline term

- gam = LinearGAM(s(0, n_splines=6)).fit(X, y)
- number of splines (n_splines) in a GAM directly controls the flexibility of the smooth function fitted to the data.

How GAM Fits the Curve

- Splines = local polynomial functions (like mini-curves) placed across the range of X
GAM finds weights for these splines to best fit the data
The final curve is a smooth sum of these spline functions

How to Interpret a Spline Curve in GAM

1. Positive or Negative Effect

If the curve **goes upward**, increasing X_i increases Y

If it **goes downward**, increasing X_i decreases Y

2. **Flat Regions**- Means X_i has **little or no effect** on Y in that range

3. **Peaks and Valleys**- Indicates **nonlinear patterns**

e.g., A peak means that there is an **optimal value** of X_i

4. Threshold Effects

If the curve is flat and then suddenly increases/decreases, it suggests a **cutoff point** beyond which the variable matters

What is a GAM?

- GAM = Generalized Additive Model
- Flexible extension of Generalized Linear Models (GLMs)
- Models the response as a sum of smooth functions of predictors
- Formula: $g(E[Y]) = \beta_0 + f_1(X_1) + f_2(X_2) + \dots + f_p(X_p)$

Components of a GAM

- 1. Link Function: Transforms $E[Y]$ to linear scale
- 2. Smooth Functions $f_i(X_i)$: Non-linear relationships
- 3. Additive Structure: Effects of variables are additive
- 4. No implicit interactions (must be added manually)

Why Use GAMs?

- Capture non-linear effects without losing interpretability
- Each variable's effect can be visualized separately
- Useful for feature analysis and transparent predictions
- Balance between linear models and black-box methods

Interpreting the Output

- Each $f_i(X_i)$ is a smooth curve showing effect of X_i
- Positive slope: increasing effect on target
- Negative slope: decreasing effect
- Flat region: little or no effect
- Look for non-linear patterns, thresholds, plateaus

Visualization of GAM Effects

- Each predictor has a partial dependence plot (PDP)
- Y-axis shows estimated contribution to the prediction
- Helps identify non-linear trends and inflection points
- Visualization enhances interpretability

GAM vs GLM vs Black-box Models

- GLM: Linear, easy to interpret but less flexible
- GAM: Additive, interpretable and flexible
- Black-box: High performance but low interpretability
- GAMs offer a strong middle ground

When to Use GAMs?

- Data shows non-linear relationships
- Interpretability is important
- You want insights from individual predictors
- Model needs to be transparent for stakeholders

Tools and Libraries

- Python: pyGAM, statsmodels, sklearn (with extensions)
- R: mgcv (most powerful GAM package)
- Supports regression, classification, and survival analysis

Apply Interpretability to Decision Tree

- Tree-based models split the data multiple times according to certain cutoff values in the features.
- Through splitting, different subsets of the dataset are created, with each instance belonging to one subset.
- The final subsets are called terminal or leaf nodes, and the intermediate subsets are called internal nodes or split nodes.
- To predict the outcome in each leaf node, the average outcome of the training data in this node is used.
- Trees can be used for classification and regression.
- There are various algorithms that can grow a tree.
- They differ in the possible structure of the tree (e.g., number of splits per node), the criteria for how to find the splits, when to stop splitting, and how to estimate the simple models within the leaf nodes.
- The classification and regression trees (CART) algorithm is probably the most popular algorithm for tree induction.

- Starting from the root node, you go to the next nodes, and the edges tell you which subsets you are looking at. Once you reach the leaf node, the node tells you the predicted outcome. All the edges are connected by 'AND'.

Feature importance

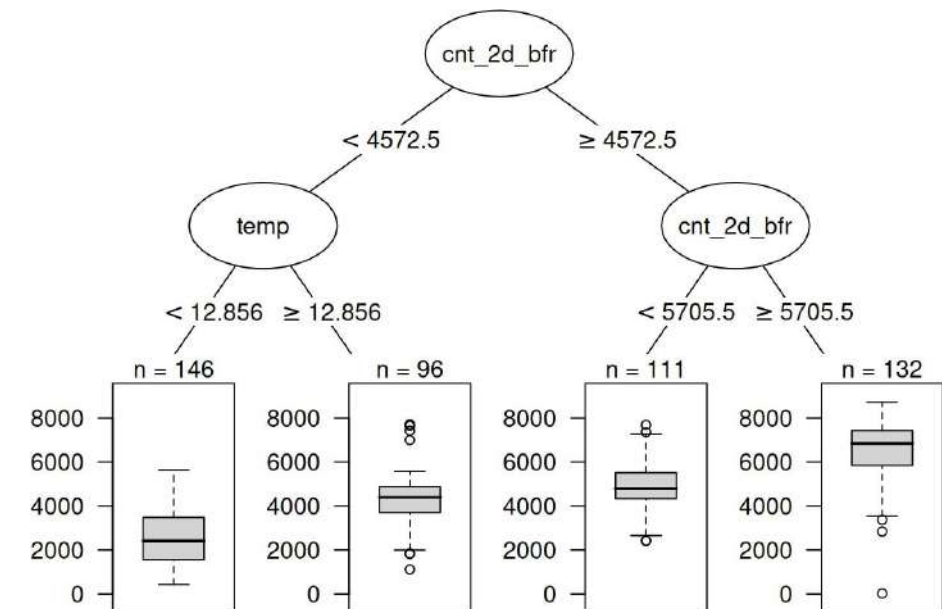
- The overall importance of a feature in a decision tree can be computed in the following way:
- Go through all the splits for which the feature was used and measure how much it has reduced the variance or Gini index compared to the parent node.
- The sum of all importances is scaled to 100.
- This means that each importance can be interpreted as a share of the overall model importance.

Tree decomposition

- Individual predictions of a decision tree can be explained by decomposing the decision path into one component per feature.
- We can track a decision through the tree and explain a prediction by the contributions added at each decision node.
- The root node in a decision tree is our starting point.
- If we were to use the root node to make predictions, it would predict the mean of the outcome of the training data.
- With the next split, we either subtract or add a term to this sum, depending on the next node in the path. To get to the final prediction, we have to follow the path of the data instance that we want to explain and keep adding to the formula.

$$\hat{f}(\mathbf{x}) = \bar{y} + \sum_{d=1}^D \text{split.contrib}(d, \mathbf{x}) = \bar{y} + \sum_{j=1}^p \text{feat.contrib}(j, \mathbf{x})$$

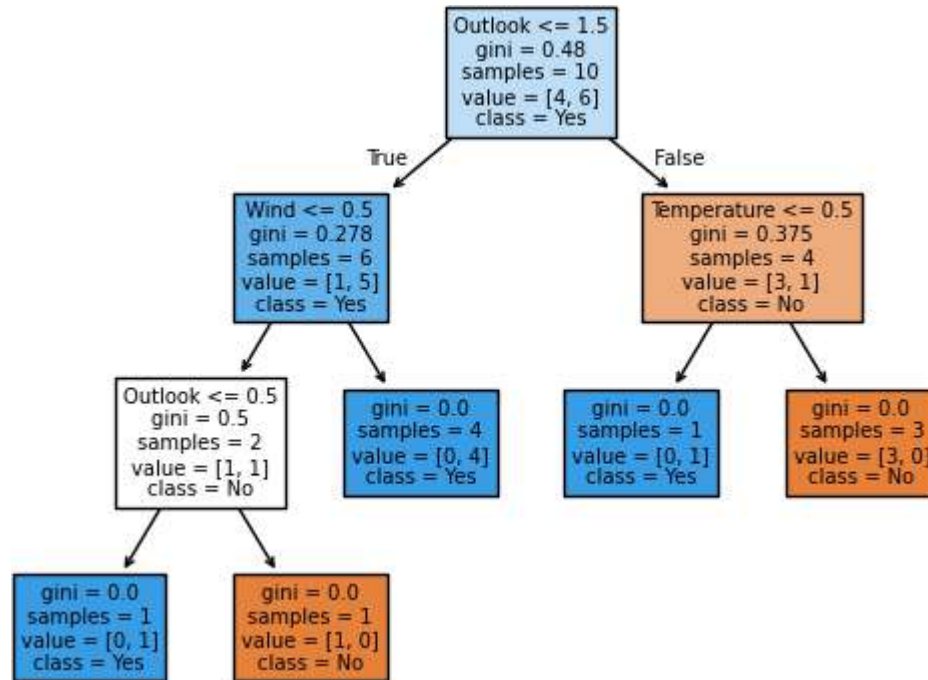
- The prediction of an individual instance is the mean of the target outcome plus the sum of all contributions of the D splits that occur between the root node and the terminal node where the instance ends up.
- A feature might be used for more than one split or not at all. We can add the contributions for each of P the features and get an interpretation of how much each feature has contributed to a prediction.



Classification model to Predict Yes/No for Playing Tennis

Sample dataset

```
data = {  
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast',  
    'Sunny', 'Sunny', 'Rain'],  
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild',  
    'Cool', 'Mild'],  
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High',  
    'Normal', 'Normal'],  
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak',  
    'Weak', 'Weak'],  
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes']  
}
```



- The root splits on Outlook ≤ 1.5 . There are 10 total samples: 4 are No, 6 are Yes. Since most samples are Yes, this node predicts Yes. Gini of 0.48 means the node is somewhat impure (not perfectly separated)

Left Branch (True: Outlook ≤ 1.5)

Next split is Wind ≤ 0.5

Then:

- If Wind ≤ 0.5 , go left $\rightarrow$ evaluate Outlook ≤ 0.5 again (indicates encoded variables reused)
- If Wind > 0.5 , go right $\rightarrow$ pure class **Yes**

Right Branch (False: Outlook > 1.5)- Split on Temperature ≤ 0.5 . Leads to two leaf nodes: One where all 3 samples are No. and one where 1 sample is Yes

Consider the test instance

Outlook	Temperature	Humidity	Wind
Rain (1)	Mild (2)	High (0)	Weak (0)

Encoded as: [1, 2, 0, 0]

Outlook $\leq 1.5 \rightarrow 1 \leq 1.5 \rightarrow \text{True} \rightarrow \text{Go Left}$

Wind $\leq 0.5 \rightarrow 0 \leq 0.5 \rightarrow \text{True} \rightarrow \text{Go Left}$

Outlook $\leq 0.5 \rightarrow 1 \leq 0.5 \rightarrow \text{False} \rightarrow \text{Go Right}$

Leaf node: gini = 0.0, value = [1, 0], class = No

Prediction: No

Neural Network Interpretation

- ✓ Learned Features
- ✓ Saliency Maps
- ✓ Detecting Concepts
- ✓ Adversarial Examples
- ✓ Influential Instances

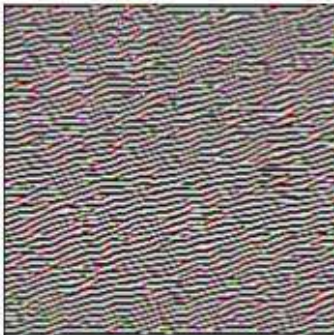
Neural Network Interpretation

Learned Features

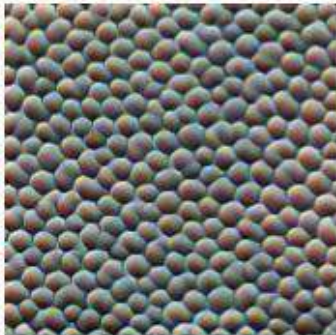
- Convolutional Neural Networks learn abstract features and concepts from raw image pixels.
- **Feature Visualization** visualizes the learned features by activation maximization.
- Network Dissection labels neural network units (e.g., channels) with human concepts.
- Deep neural networks learn high-level features in the hidden layers. This is one of their greatest strengths and reduces the need for feature engineering.

- With convolutional neural networks, the image is fed into the network in its raw form (pixels). The network transforms the image many times.
- First, the image goes through many convolutional layers.
- In those convolutional layers, the network learns new and increasingly complex features in its layers.
- Then the transformed image information goes through the fully connected layers and turns into a classification or prediction.
- The first convolutional layer(s) learn features such as edges and simple textures.
- Later convolutional layers learn features such as more complex textures and patterns.
- The last convolutional layers learn features such as objects or parts of objects.
- The fully connected layers learn to connect the activations from the high-level features to the individual classes to be predicted.

Edges



Textures



Patterns



Parts



Objects



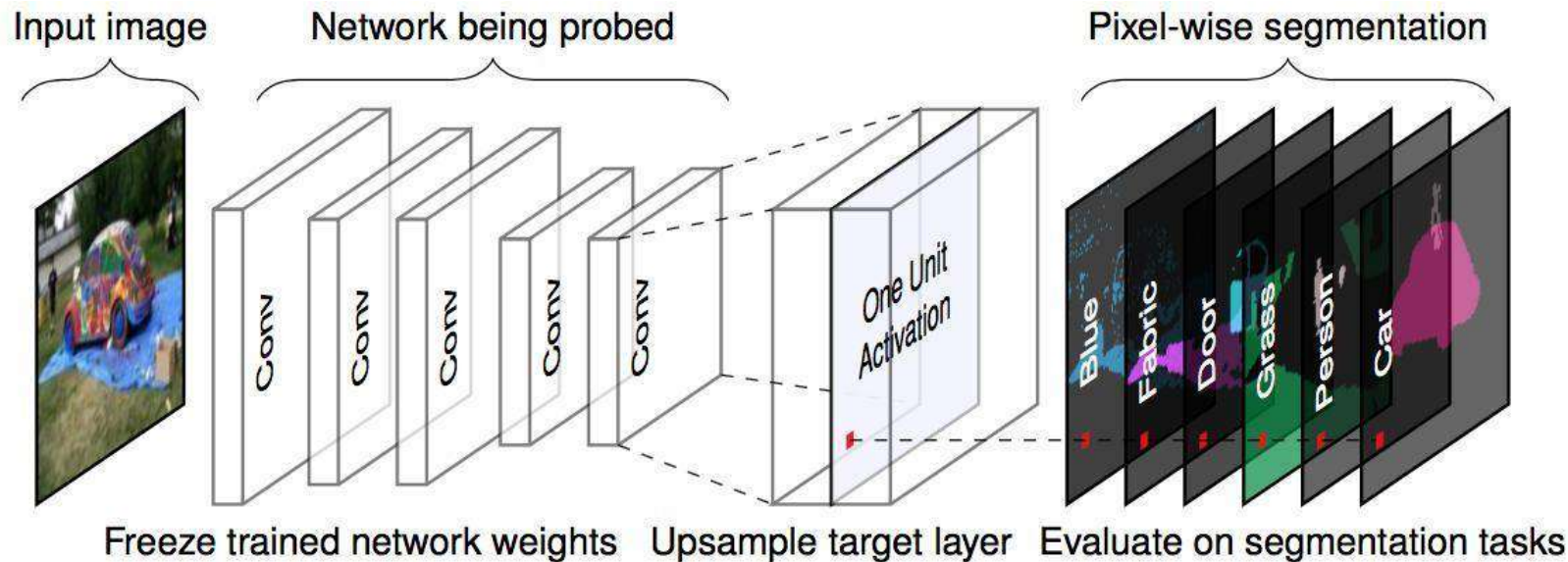
Feature visualization

- The approach of making the learned features explicit is called Feature Visualization.
- Feature visualization for a unit of a neural network is done by finding the input that maximizes the activation of that unit.
- “Unit” refers either to individual neurons, channels (also called feature maps), entire layers, or the final class probability in classification

Network Dissection

Network Dissection has three steps:

- Get images with human-labeled visual concepts, from stripes to skyscrapers.
- Measure the CNN channel activations for these images.
- Quantify the alignment of activations and labeled concepts.



- The networks detect lower-level concepts (colors, textures) at lower layers and higher-level concepts (parts, objects) at higher layers.
- Batch normalization reduces the number of unique concept detectors.
- Many units detect the same concept.
- Increasing the number of channels in a layer increases the number of interpretable units.

Technique	Description	Use Case	Type
Activation Maps	Visualize intermediate outputs (feature maps) from CNN layers	Images (CNNs)	Global
Activation Maximization	Optimize input to maximize activation of a specific neuron or filter	Images, NLP	Global
Deconvolution	Projects activations back to input space	Images	Global
Feature Inversion	Reconstruct input that caused a certain activation	Images	Local/Global

Technique	Description	Use Case	Type
Saliency Maps	Highlights input areas with highest gradient	Images	Local
Grad-CAM	Uses gradients and feature maps to localize important regions	Images	Local
Guided Backpropagation	Combines saliency and deconvolution for sharper explanations	Images	Local
Integrated Gradients	Averages gradients along the path from baseline input	All	Local

Pixel attribution methods

- **Pixel attribution methods** highlight the pixels that were relevant for a certain image classification by a neural network.
- Pixel attribution methods can be found under various names: sensitivity map, saliency map, pixel attribution map, gradient-based attribution methods, feature relevance, feature attribution, and feature contribution.
- Pixel attribution is a special case of feature attribution, but for images.
- Feature attribution explains individual predictions by attributing each input feature according to how much it changed the prediction (negatively or positively).
- The features can be input pixels, tabular data, or words.
- SHAP, Shapley values, and LIME are examples of general feature attribution methods.

Pixel attribution approaches

- **Occlusion or perturbation-based:** Methods like SHAP and LIME manipulate parts of the image to generate explanations (model-agnostic).
- **Gradient-based:** Many methods compute the gradient of the prediction (or classification score) with respect to the input features.
- The gradient-based methods mostly differ in how the gradient is computed.

Gradient-only methods :

- Gradient-only methods tell us whether a change in a pixel would change the prediction.
- Examples are Vanilla Gradient and Grad-CAM .
- The interpretation of the gradient-only attribution is: If I were to increase the color values of the pixel, the predicted class probability would go up (for positive gradient) or down (for negative gradient).
- The larger the absolute value of the gradient, the stronger the effect of a change of this pixel.

Path-attribution methods:

- Path-attribution methods compare the current image to a reference image, which can be an artificial “zero” image such as a completely gray image.
- The difference in actual and baseline prediction is divided among the pixels.
- The baseline image can also be multiple images: a distribution of images.
- This category includes model-specific gradient-based methods such as Deep Taylor and Integrated Gradients as well as model-agnostic methods such as LIME and SHAP.
- Some path-attribution methods are “complete,” meaning that the sum of the relevance scores for all input features is the difference between the prediction of the image and the prediction of a reference image.
- Examples are SHAP and Integrated Gradients.
- For path-attribution methods, the interpretation is always done with respect to the baseline: The difference between classification scores of the actual image and the baseline image is attributed to the pixels.

Grad-CAM

- Grad-CAM stands for Gradient-weighted Class Activation Map.
- Grad-CAM provides visual explanations for CNN decisions. it's based on the gradient of the neural networks.
- Gradient is not backpropagated all the way back to the image, but (usually) to the last convolutional layer to produce a coarse localization map that highlights important regions of the image.
- Grad-CAM, like other techniques, assigns each neuron a relevance score for the decision of interest. This decision of interest can be the class prediction (which we find in the output layer), but can theoretically be any other layer in the neural network. Grad-CAM backpropagates this information to the last convolutional layer. Grad-CAM can be used with different CNNs: with fully connected layers, for structured output such as captioning and in multi-task outputs, and for reinforcement learning.

Detecting Concepts

- A concept can be any abstraction, such as a color, an object, or even an idea.
- Given any user-defined concept, although a neural network might not be explicitly trained with the given concept, the concept-based approach detects that concept embedded within the latent space learned by the network.
- The concept-based approach can generate explanations that are not limited by the feature space of a neural network.

Evaluation of Interpretability Methods

Levels of evaluation

- ✓ Application level evaluation
- ✓ Human level evaluation
- ✓ Function level evaluation

Application level evaluation (real task):

Application level evaluation (real task):

- Put the explanation into the product and have it tested by the end user.
- Imagine fracture detection software with a machine learning component that locates and marks fractures in X-rays.
- At the application level, radiologists would test the fracture detection software directly to evaluate the model.
- This requires a good experimental setup and an understanding of how to assess quality.
- A good baseline for this is always how good a human would be at explaining the same decision.

Human level evaluation (simple task)

- **Human level evaluation** (simple task) is a simplified application level evaluation.
- The difference is that these experiments are not carried out with the domain experts, but with laypersons. This makes experiments cheaper and it is easier to find more testers.
- An example would be to show a user different explanations, and the user would choose the best one.

Function level evaluation (proxy task)

- **Function level evaluation** (proxy task) does not require humans.
- This works best when the class of model used has already been evaluated by someone else in a human level evaluation.
- For example, it might be known that the end users understand decision trees. In this case, a proxy for explanation quality may be the depth of the tree. Shorter trees would get a better explainability score. It would make sense to add the constraint that the predictive performance of the tree remains good and does not decrease too much compared to a larger tree.

Properties of Explanation Methods

- Expressive Power
- Translucency
- Portability
- Algorithmic Complexity

Expressive Power

- Expressive Power is the “language” or structure of the explanations the method is able to generate. An explanation method could generate IF-THEN rules, decision trees, a weighted sum, natural language, or something else.

Translucency

- Translucency describes how much the explanation method relies on looking into the machine learning model, like its parameters. For example, explanation methods relying on intrinsically interpretable models like the linear regression model (model-specific) are highly translucent.
- Methods only relying on manipulating inputs and observing the predictions have zero translucency.
- Depending on the scenario, different levels of translucency might be desirable.
- The advantage of high translucency is that the method can rely on more information to generate explanations. The advantage of low translucency is that the explanation method is more portable.

Portability

- Portability describes the range of machine learning models with which the explanation method can be used.
- Methods with a low translucency have a higher portability because they treat the machine learning model as a black box.
- Surrogate models might be the explanation method with the highest portability. Methods that only work for e.g., recurrent neural networks have low portability.

Algorithmic Complexity

- Algorithmic Complexity describes the computational complexity of the method that generates the explanation.
- This property is important to consider when computation time is a bottleneck in generating explanations.

Properties of Individual Explanations

- Accuracy
- Fidelity
- Consistency
- Stability
- Comprehensibility
- Certainty
- Degree of Importance
- Novelty
- Representativeness

Accuracy:

- How well does an explanation predict unseen data?
- High accuracy is especially important if the explanation is used for predictions in place of the machine learning model.
- Low accuracy can be fine if the accuracy of the machine learning model is also low, and if the goal is to explain what the black box model does.

Fidelity:

- How well does the explanation approximate the prediction of the black box model?
- High fidelity is one of the most important properties of an explanation because an explanation with low fidelity is useless to explain the machine learning model.
- Accuracy and fidelity are closely related. If the black box model has high accuracy and the explanation has high fidelity, the explanation also has high accuracy.
- Some explanations offer only local fidelity, meaning the explanation only approximates well to the model prediction for a subset of the data (e.g., local surrogate models) or even for only an individual data instance (e.g., Shapley Values).

Consistency:

- How much does an explanation differ between models that have been trained on the same task and that produce similar predictions?
- For example, train a support vector machine and a linear regression model on the same task, and assume both produce very similar predictions.
- Compute explanations using a method of your choice and analyze how different the explanations are.
- If the explanations are very similar, the explanations are highly consistent.

Stability:

- How similar are the explanations for similar instances?
- While consistency compares explanations between models, stability compares explanations between similar instances for a fixed model.
- High stability means that slight variations in the features of an instance do not substantially change the explanation (unless these slight variations also strongly change the prediction).
- A lack of stability can be the result of a high variance of the explanation method..

Comprehensibility:

- How well do humans understand the explanations?
- Comprehensibility depends on the audience. Ideas for measuring comprehensibility include measuring the size of the explanation (number of features with a non-zero weight in a linear model, number of decision rules, ...) or testing how well people can predict the behavior of the machine learning model from the explanations.
- The comprehensibility of the features used in the explanation should also be considered.
- A complex transformation of features might be less comprehensible than the original features.

Certainty:

Does the explanation reflect the certainty of the machine learning model?

- Many machine learning models only give predictions without a statement about the model's confidence that the prediction is correct.
- If the model predicts a 4% probability of cancer for one patient, is it as certain as the 4% probability that another patient, with different feature values, received?
- An explanation that includes the model's certainty is very useful. In addition, the explanation itself may be a model or an estimate based on data and therefore itself subject to uncertainty.
- This uncertainty of the explanation is also a relevant property of the explanation itself.

Degree of Importance:

How well does the explanation reflect the importance of features or parts of the explanation?

- For example, if a decision rule is generated as an explanation for an individual prediction, is it clear which of the conditions of the rule was the most important?

Novelty: Does the explanation reflect whether a data instance to be explained comes from a “new” region far removed from the distribution of training data?

- In such cases, the model may be inaccurate and the explanation may be useless. The concept of novelty is related to the concept of certainty. The higher the novelty, the more likely it is that the model will have low certainty due to lack of data.

Representativeness:

- How many instances does an explanation cover?
- Explanations can cover the entire model (e.g., interpretation of weights in a linear regression model) or represent only an individual prediction (e.g., Shapley Values).

Interpretability – GAM-

Additional Examples

GAM – Generalized Additive Model

- A **GAM** models the expected value of a response variable as a sum of smooth functions of predictor variables:

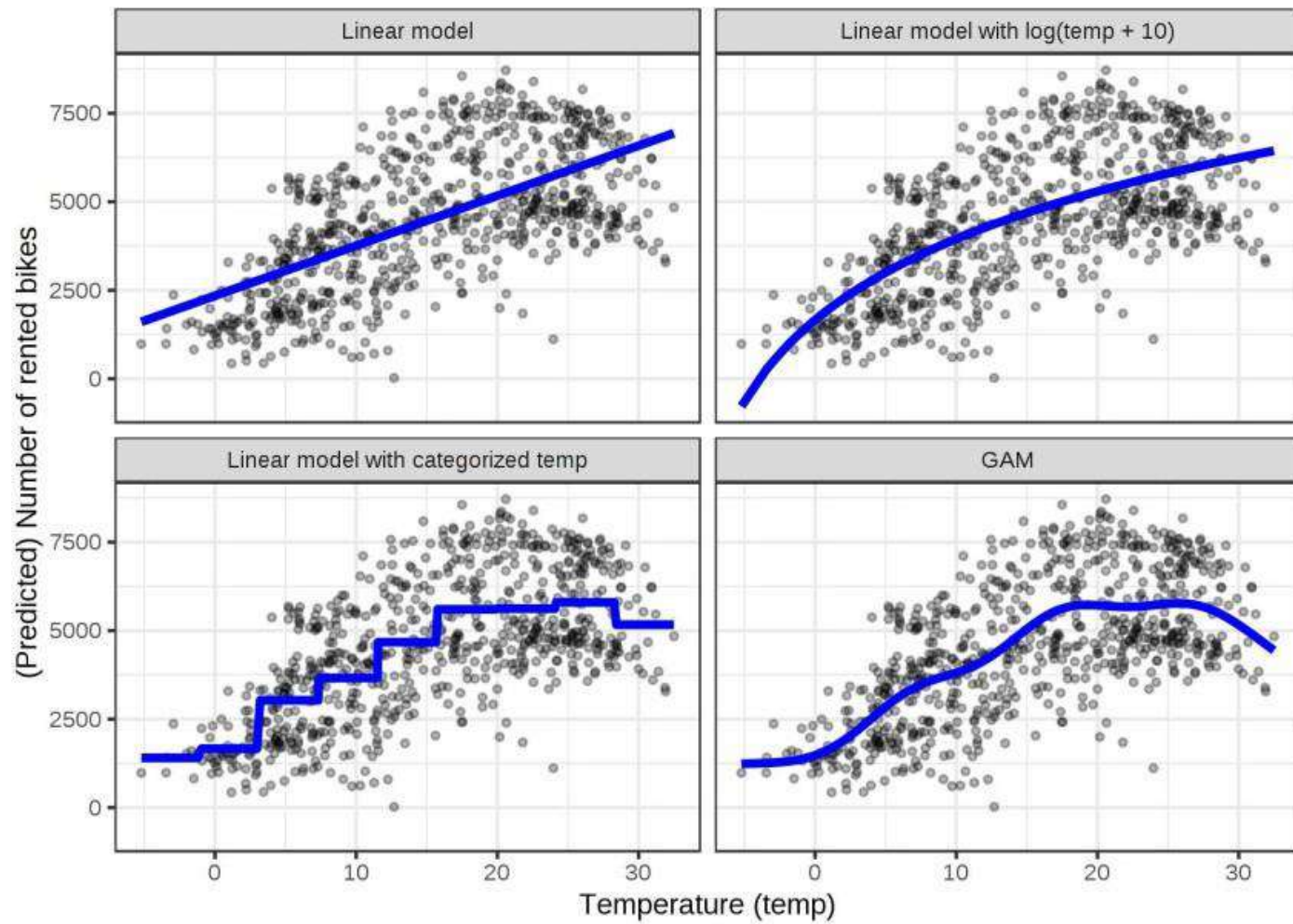
$$g(E(Y)) = \beta_0 + f_1(x_1) + f_2(x_2) + \dots + f_p(x_p) + \varepsilon$$

- **g**: Link function (e.g., logit, identity)
- **$f_1(x_1)$** : Smooth non linear functions of predictor variable x
- **β_0** : Intercept
- **ε** : error term

GAM – Generalized Additive Model

Example: Bike rentals

- Features – Temperature, O/P number of bikes rented
- Effect on the number of rental bikes as increasing the temperature by 1 degree at 10 to 11 degree. The positive effect.
- Effect on bicycle rentals and from 40 to 41 may be negative.
- How should be linear regression fitting here?



GAM – Generalized Additive Model

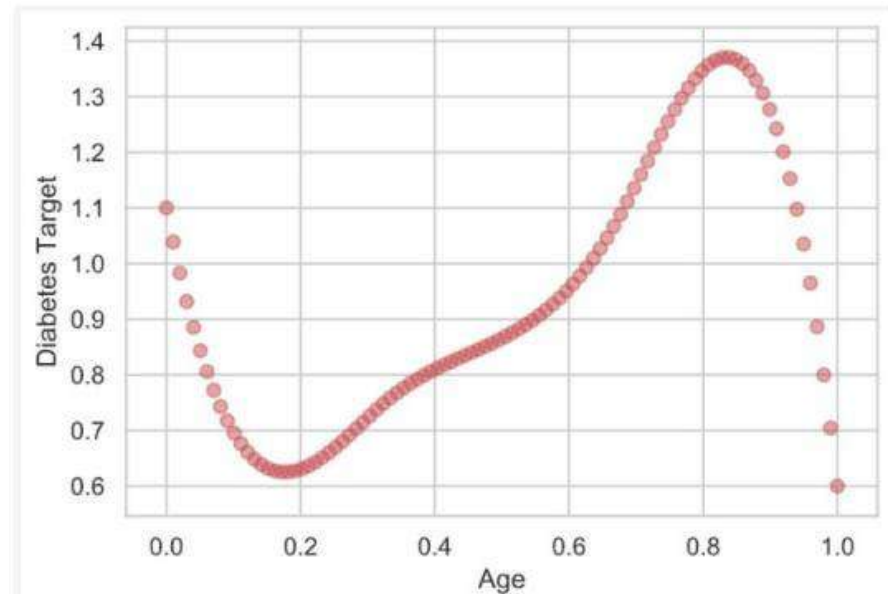
- Non-linear relationship between age and the target variable
- I/P (Age) to target (diabetes)
- Linear vs. polynomial

$$y = w_0 + w_1x_1 \text{ (Degree 1)}$$

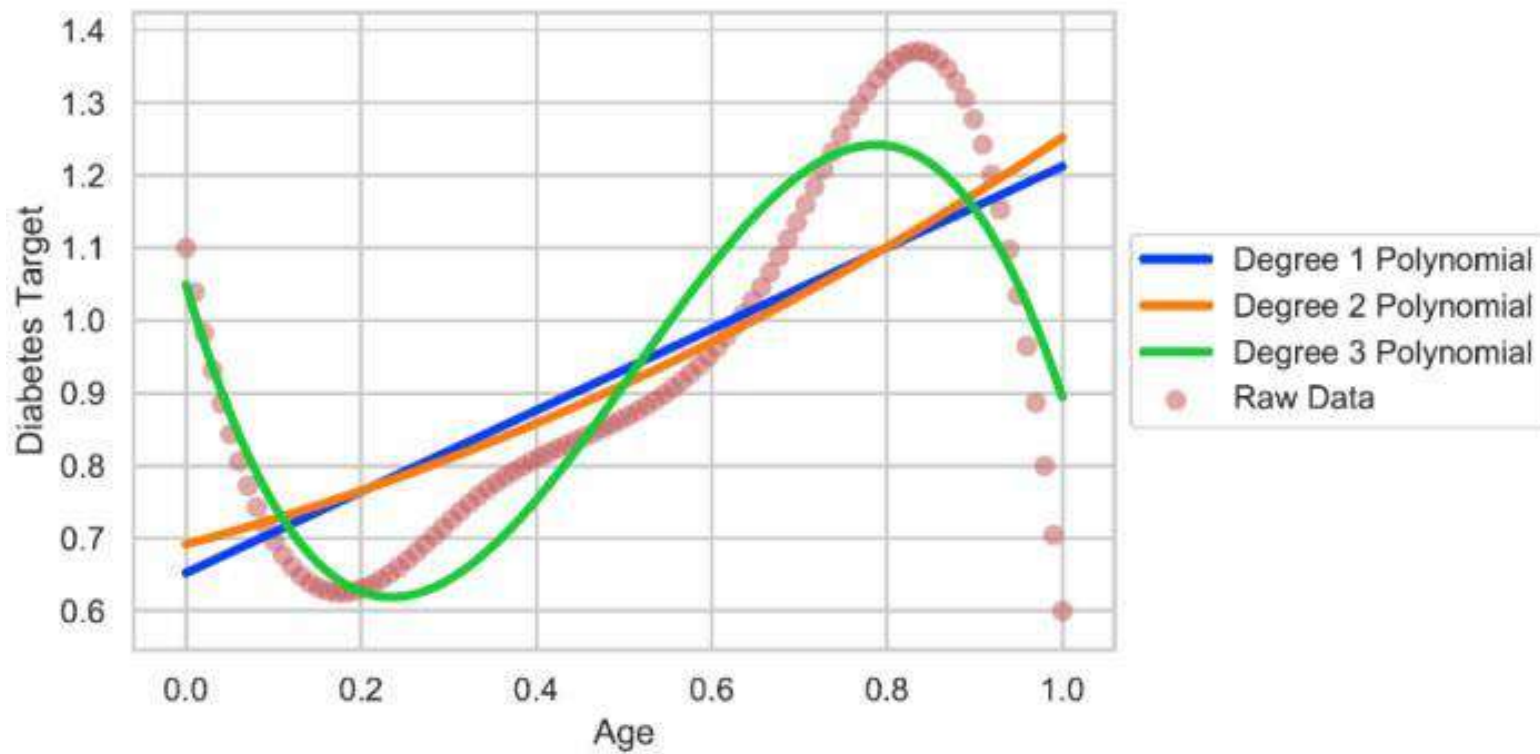
$$y = w_0 + w_1x_1 + w_2x_1^2 \text{ (Degree 2)}$$

$$y = w_0 + w_1x_1 + w_2x_1^2 + w_3x_1^3 \text{ (Degree 3)}$$

- Complex with more features
 - Value of polynomial vs. feature
- Overfitting
- Transformation of feature by logarithm, square root
- Categorical transformation as in decision tree



Polynomial - function



GAM – Generalized Additive Model

- Non-linear relationships are modeled using
 - smoothing functions for each feature
 - adding all of them.
 - each feature has its own associated smoothing function that best models the relationship between that feature and the target.

$$y = w_0 + \underbrace{f_1(x_1)}_{\substack{\text{Smoothing Function} \\ \text{for Feature } x_1}} + \underbrace{f_2(x_2)}_{\substack{\text{Smoothing Function} \\ \text{for Feature } x_2}} + \dots + \underbrace{f_n(x_n)}_{\substack{\text{Smoothing Function} \\ \text{for Feature } x_n}}$$

Features of GAM

- **Flexible modeling** of non-linear relationships.
- **Additive structure** makes interpretation easier (sum of functions of predictor variable).
- Can handle **different types of response variables** (binary, count, continuous).
- **Interpretability** - visualize relationship between each predictor and response variable using **smoothing techniques** like splines or loess.
- **Tools:** R, python, visualization plots

GAM – Generalized Additive Model

- **regression splines**

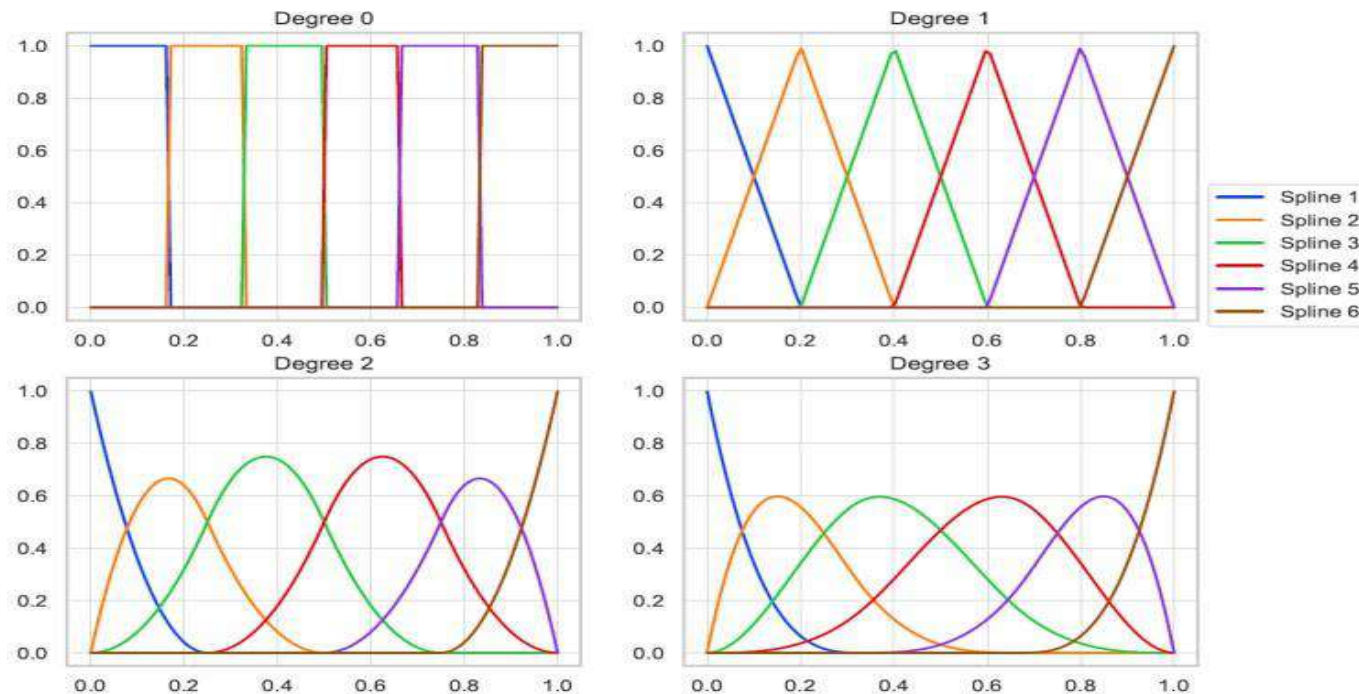
- widely used smoothing function
- practical and computationally efficient.
- A weighted sum of linear or polynomial functions.
- These polynomial functions are also known as basis functions.
- w_k – weight of function
- b_k – base function
- A basis function is a family of transformations that can be used to capture a general shape or non-linear relationship.

$$f_j(x_j) = \underbrace{\sum_{k=1}^K w_k b_k(x_j)}_{\text{Smoothing Function represented as a weighted sum of basis functions}}$$

GAM – Generalized Additive Model

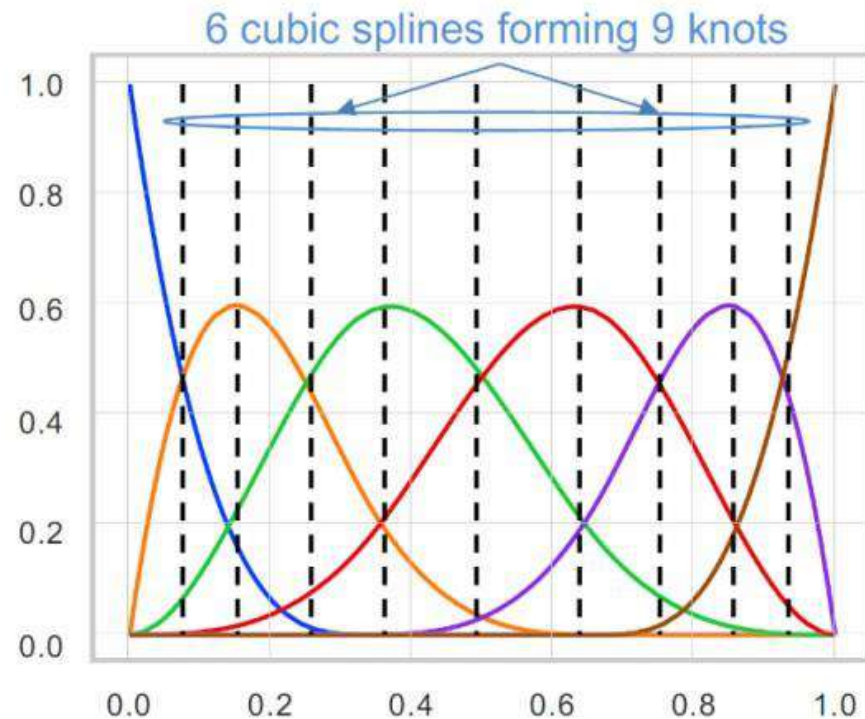
- **splines**

- A spline is a polynomial of degree with continuous derivatives.



GAM

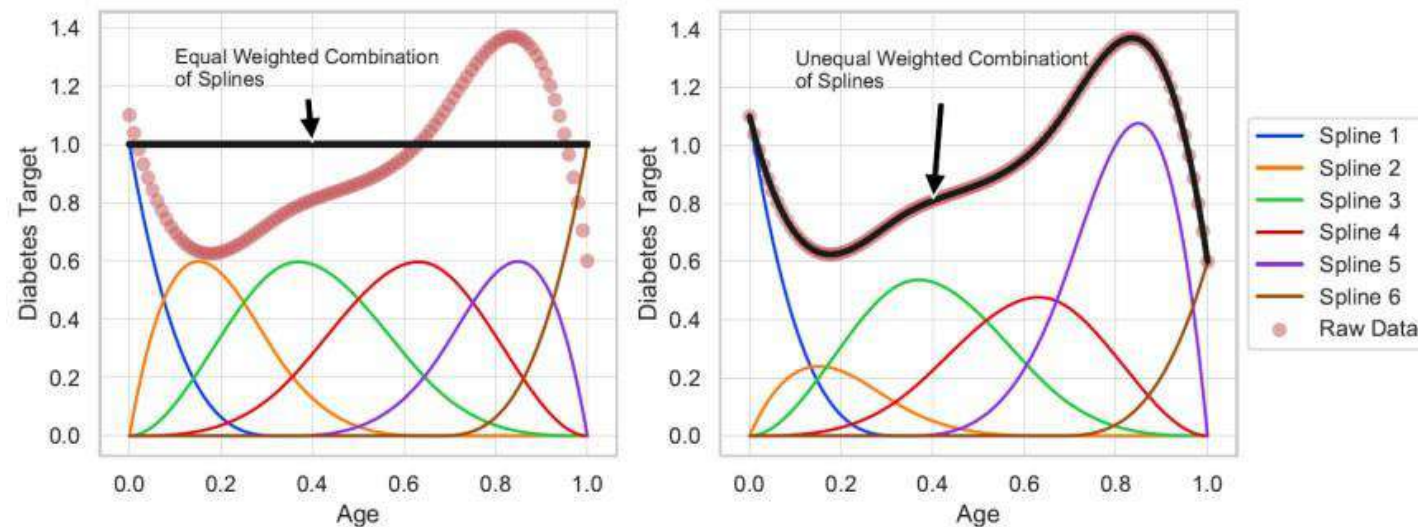
- Python package – pyGAM
- A regression spline is
 - A weighted sum of basis functions or splines.
 - This is essentially a linear regression problem to learn the weights.
 - Use method of least squares and gradient descent.
 - Need to specify hyperparameter => the number of knots or degrees of freedom.
 - K splines will have K+3 knots/degree of freedom
 - Determine hyperparameter using **cross-validation**.



GAM – Generalized Additive Model

- **splines**

- **degree 3** or **cubic splines** are used as it can capture most general shapes.
- to capture a general shape => take a weighted sum of the splines



GAM

- Need for regularization
- In GAMs, you can easily overfit
 - increasing the number of splines or degrees of freedom.
 - If the number of splines is high,
 - the resulting smoothing function => a weighted sum of the splines overfits to noise in data
 - Add regularization to cost function
 - the integral of the squared of the 2nd order derivative of the function.
 - This is one hyperparameter to be resolved using cross-validation.

Interpretation on smooth functions

Each term $f_i(X_i)$ shows how the predictor X_i affects the response:

- **Shape of the curve:** Indicates the nature of the relationship (e.g., linear, U-shaped, plateau).
- **Vertical position:** Shows the magnitude of the effect.
- **Effective degrees of freedom (EDF):** Higher EDF means more flexibility (complexity) in the curve.

Interpreting GAM - Visualize

- **Visualize the effects of each feature on the target**
 - Partial dependence or partial effects plots.
 - Partial dependence looks at the effect of each feature by marginalizing on the rest.
 - It is highly interpretable as we can see the average effect of each feature value on the target variable.
 - We can see if the target response to the feature is linear, non-linear, monotonic or non-monotonic.

Limitations of GAM

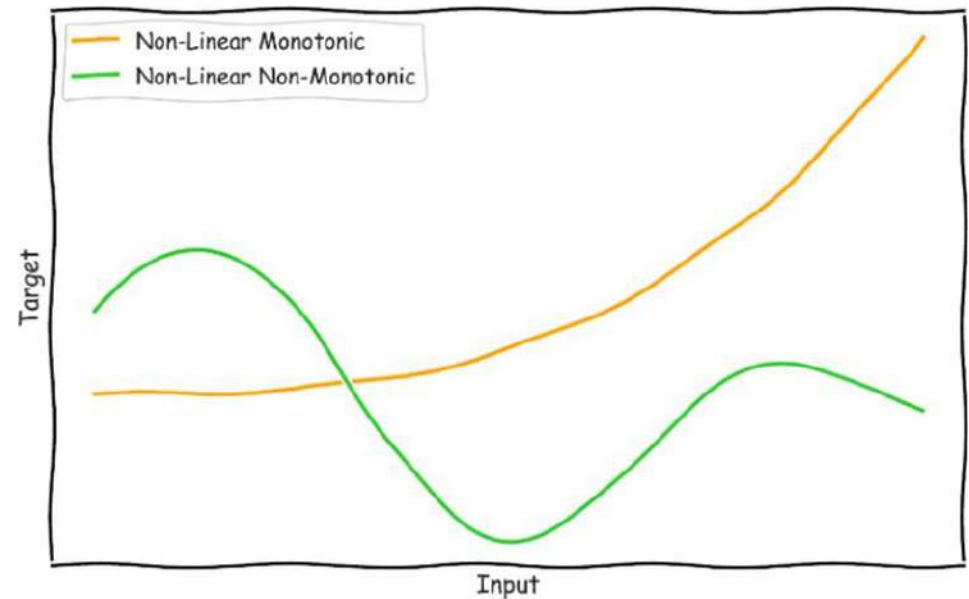
- GAMs have a tendency to **overfit**, but can be resolved with regularization.
- **Sensitive to feature values outside of the range** in the training set
 - May lose its predictive power when exposed to outlier values.
- For critical tasks => Limited predictive power of GAM needs blackbox ML/DL models

Decision Tree

Decision trees can be

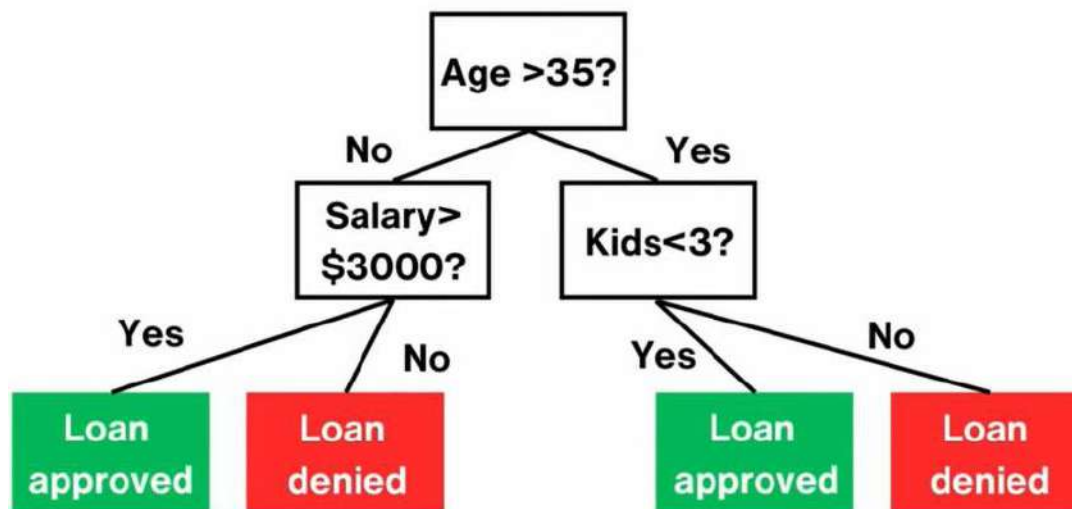
- used to model complex nonlinear relationships.
 - Non-linear monotonic
 - Non-linear non-monotonic
- applied for both
 - Regression (MSE)
 - Classification (Gini impurity, Entropy).
- Highly interpretable

A decision tree is to find optimum splits in the data that best predict the output or target variable



Interpretation on Decision Tree

Should loan be approved or denied?



Age: 32

Salary: \$2800

Kids: 2

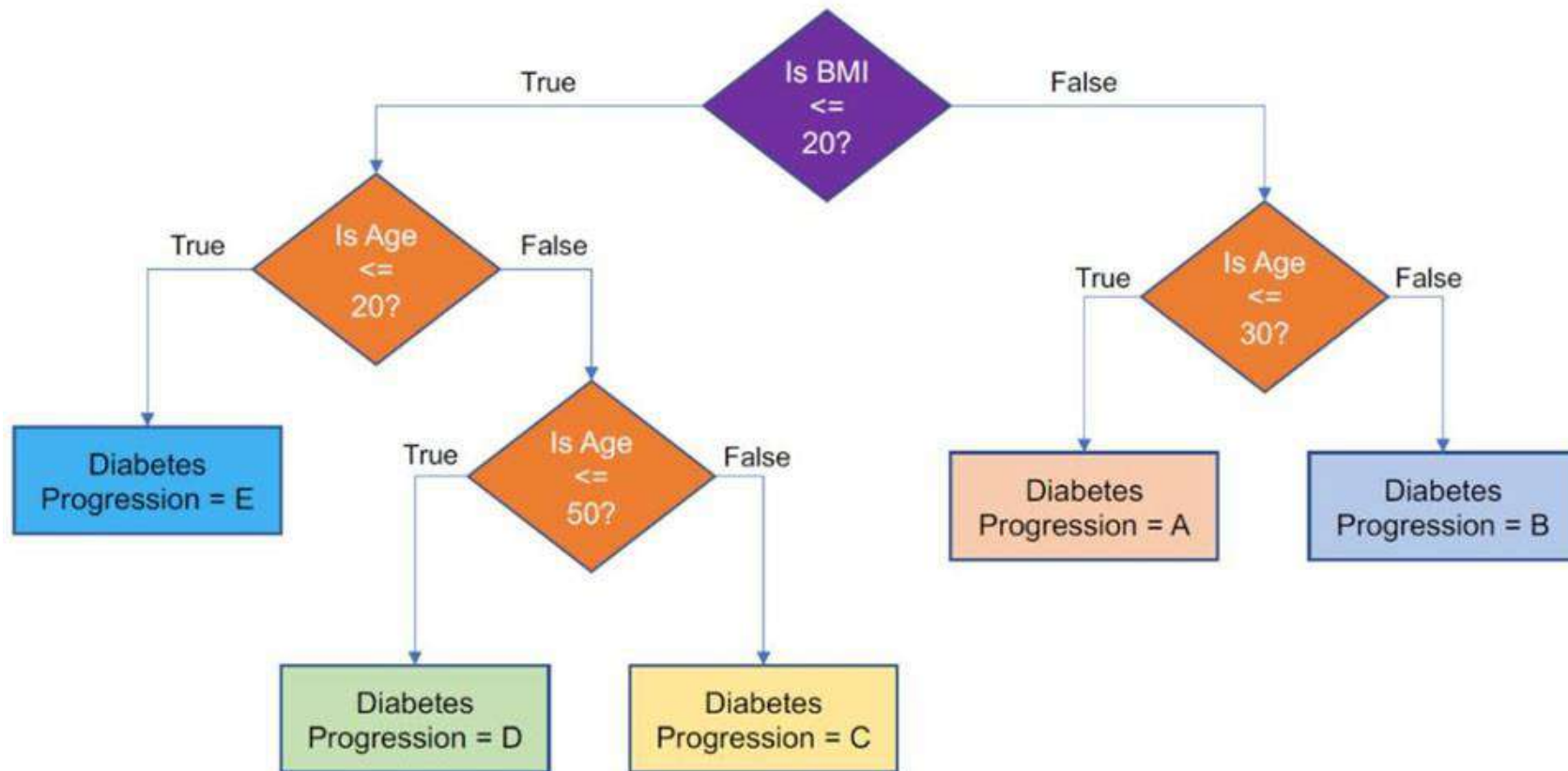


Loan denied

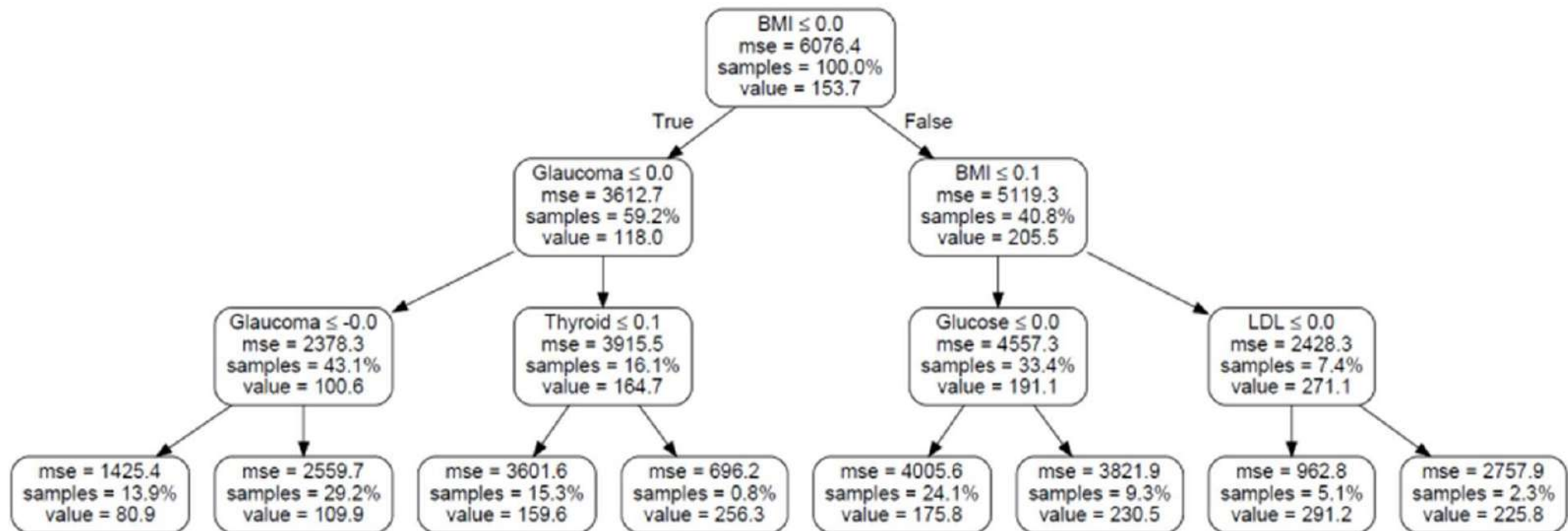
Decision Tree

- Model is **interpretable by design**
 - Decision of loan denied is clearly known
- Suggest user to avail loan by improving on his ineligible variable
- SHAP and LIME can improve interpretability

Example – Diabetic Prediction



Interpretability for Decision Tree



Interpretation Approaches on Decision Tree

- **Simple interpretation**

- **Template:** If feature x is [smaller/bigger] than threshold c AND ... then the predicted outcome is the mean value of y of the instances in that node.

- **Feature importance**

- Go through all the splits for which the feature was used and measure how much it has reduced the variance or Gini index compared to the parent node. The sum of all importances is scaled to 100

- **Tree decomposition**

- Decomposing the decision path into one component per feature
- Track a decision through the tree and explain a prediction by the contributions added at each decision node

Feature importance

Compute the importance of a node in the binary tree:

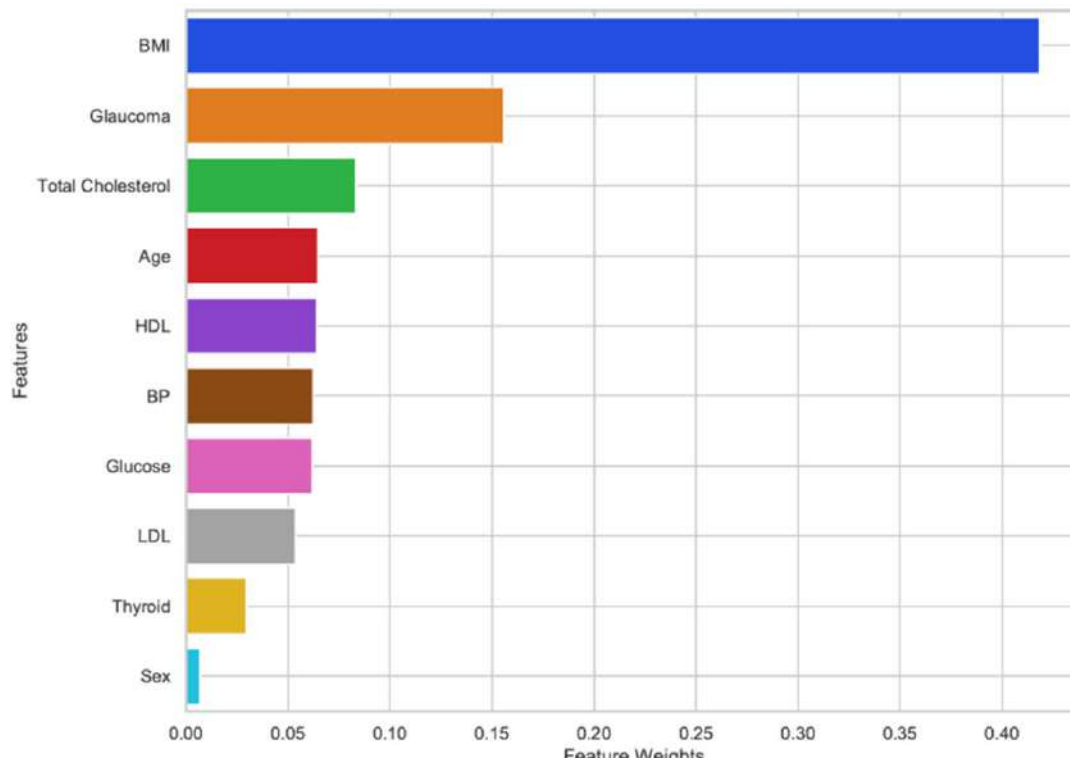
- The importance of a node is computed as
- the *decrease in the **cost function or impurity measure** for that node* weighted by the probability of reaching that node in the tree.

$$\underbrace{I_k^{\text{node}}}_{\text{Importance of node } k} = \underbrace{p_k}_{\substack{\text{Proportion of samples} \\ \text{to reach node } k}} \cdot \underbrace{m_k}_{\substack{\text{Impurity measure} \\ \text{of node } k}} - \underbrace{p_k^{(\text{left})}}_{\substack{\text{Proportion of samples} \\ \text{to reach left subtree} \\ \text{of node } k}} \cdot \underbrace{m_k^{(\text{left})}}_{\substack{\text{Impurity measure} \\ \text{of left subtree of} \\ \text{node } k}} - \underbrace{p_k^{(\text{right})}}_{\substack{\text{Proportion of samples} \\ \text{to reach right subtree} \\ \text{of node } k}} \cdot \underbrace{m_k^{(\text{right})}}_{\substack{\text{Impurity measure} \\ \text{of right subtree of} \\ \text{node } k}}$$

- Summing up the importance of the nodes that split on that feature **normalized** by the importance of all the nodes to 100.

$$\underbrace{I_i^{\text{feature}}}_{\text{Importance of feature } i} = \frac{\overbrace{\sum_{j \in \mathbb{J}} I_j^{\text{node}}}^{\text{Sum of importance of all nodes } j \text{ that split on feature } i}}{\underbrace{\sum_{k \in \mathbb{K}} I_k^{\text{node}}}_{\text{Sum of importance of all nodes } k \text{ in the decision tree}}}$$

Feature importance => a higher value implies greater importance



- BMI => 42%
- Glaucoma ?
- Gender lowest
- Prune the feature with lowest importance to Improve performance of Model.

Example- Predicting Loan Approval

Feature Importance:

- **Credit Score** is the most influential feature in determining loan approval.
- **Income** and **Loan Amount** also play significant roles.
- **Employment Status** has minimal impact in this tree.

Feature	Importance Score (Gini score)
Credit Score	0.45
Income	0.30
Loan Amount	0.20
Employment Status	0.05

Example

Tree Decomposition

if Credit Score > 700:
 if Income > ₹50,000:
 Predict: Approved else: Predict: Denied
else:
 if Loan Amount < ₹5,00,000:
 Predict: Approved else: Predict: Denied

Interpretation:

- The tree first splits on **Credit Score** - most critical feature.
- If the score is high, **Income** becomes the deciding factor.
- If the score is low, **Loan Amount** is considered.
- Each path leads to a leaf node with a prediction (Approved/Denied).

Challenges in Interpretation

- **Overfitting** - Deep trees are hard to interpret and may not generalize.
- **Data bias** - Decisions depend heavily on training data distribution.
- **Categorical splits** - Can be harder to visualize when many categories exist.
- **Hyperparameter tuning** – Max_depth, Minimum Sample for node

Advantages:

- Tree structure is ideal for **capturing interactions** between features in the data.
- Data ends up in **distinct groups** that are often easier to understand
- Tree structure has a **natural visualization**, with its nodes and edges.
- No need to transform features

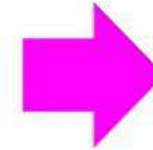
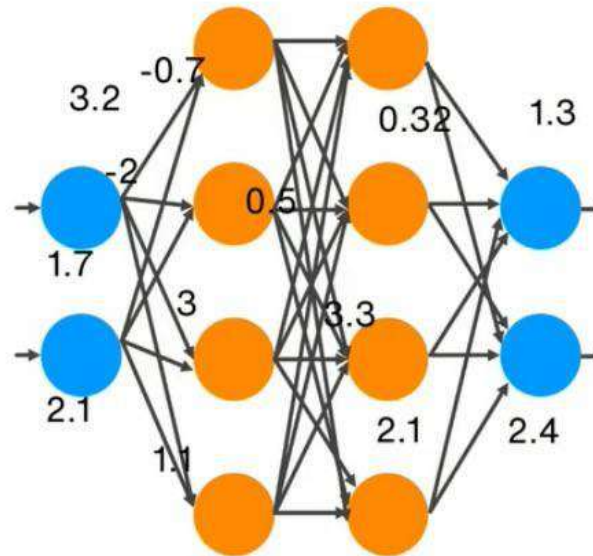
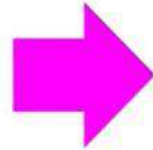
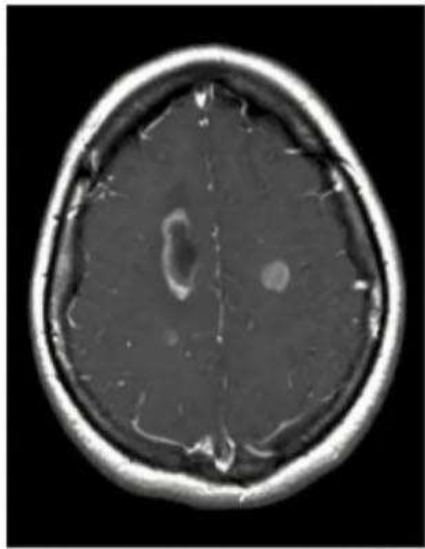
Disadvantages:

- Trees **fail** to deal with **linear** relationships.
- **lack of smoothness** - slight changes in the input feature can have a big impact on the predicted outcome
- **Unstable** - changes in the training dataset can create a completely different tree
- very interpretable— as long as they are short. **Difficult for deep tree - complicate rules**

Interpretation Properties – Decision Tree

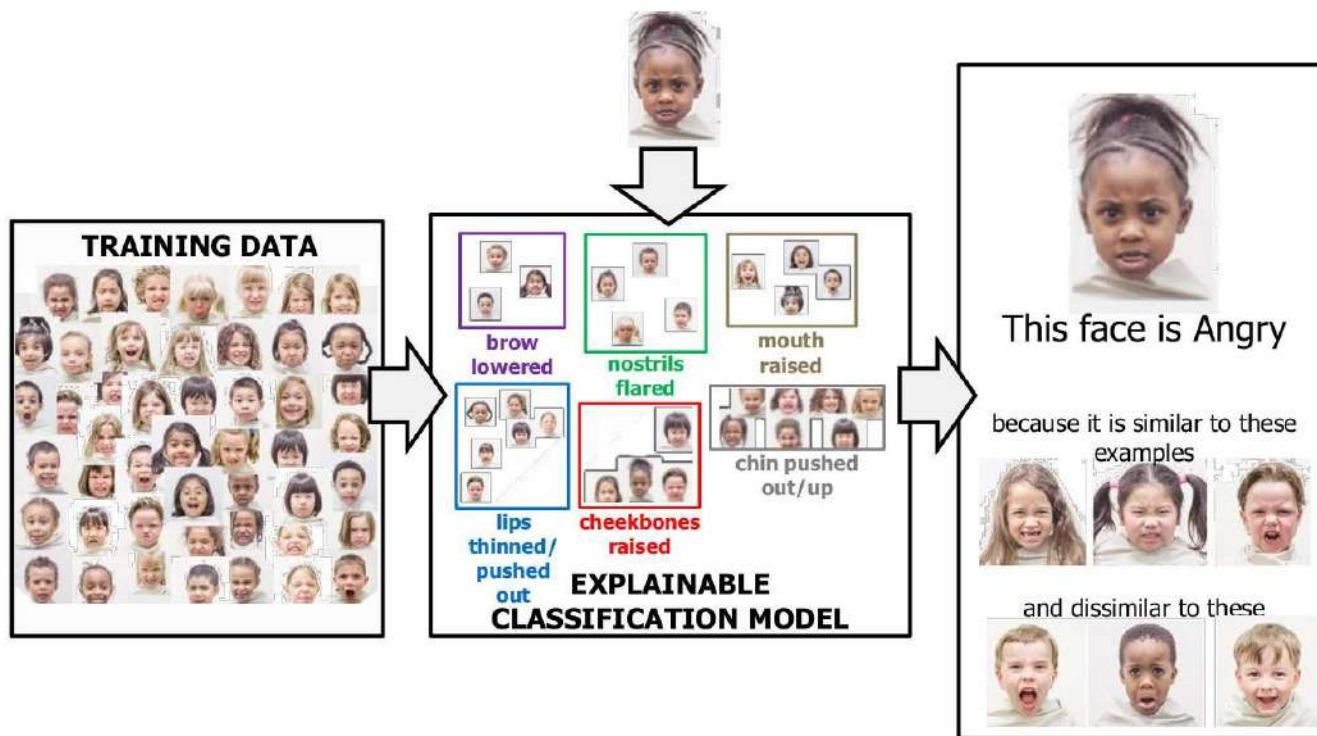
Explainable Properties	Decision Tree (CART)
Local / Global	Global
Linearity	Non- Linear
Monotonic/ Non-Monotonic	Some Monotonicity
Feature interaction captured	Yes
Model Complexity (suited)	Medium to High

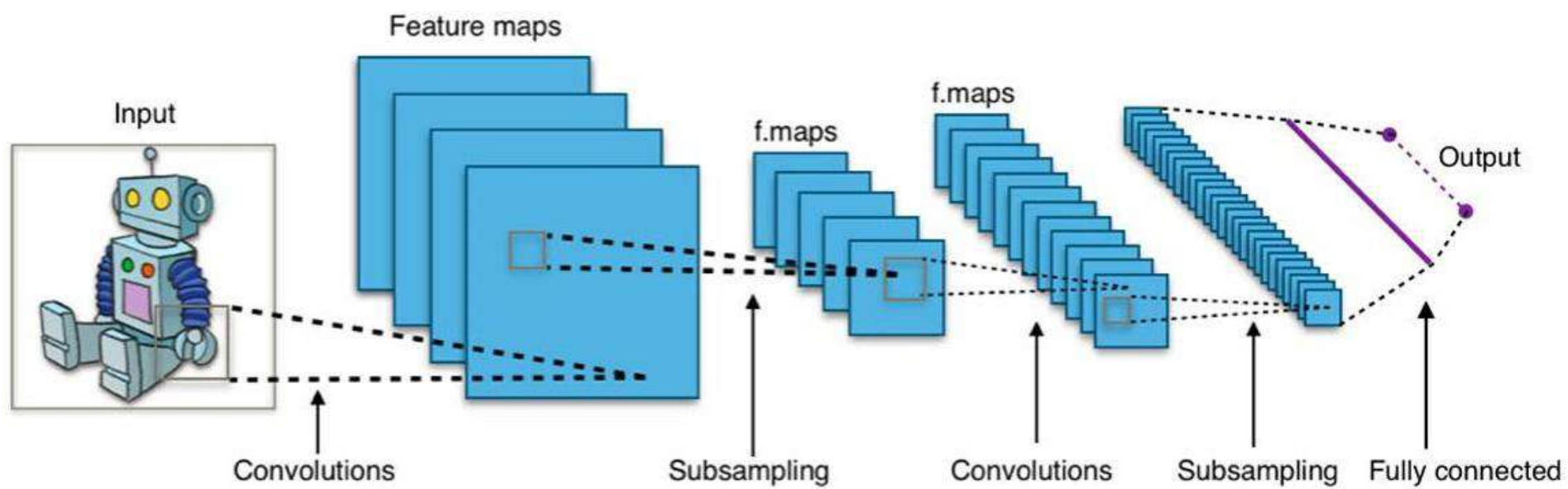
Interpretation on Neural Networks



**Tumor
detected**

- Not interpretable, since don't know exactly which feature/ pixels contributed to classify as tumour
- Hence, have to do something to make the model interpretable
- This is Post-hoc interpretability





Interpretation on NN

- **Feature Importance**

- **Permutation importance:** Measures how shuffling a feature affects model performance.
- **Integrated gradients:** Tracks how changes in input affect output across the network.

- **Activation Visualization**

- Shows how neurons respond to different inputs.
- Helps understand what patterns the network is learning.

- **Layer-wise Relevance Propagation (LRP)**

- Breaks down predictions to show which parts of the input contributed most.

- **SHAP (SHapley Additive exPlanations)**

- Assigns each feature a contribution value for a specific prediction.
- Works well with deep learning models.

- **Partial Dependence Plots**

- Show how changing one feature affects the prediction, keeping others constant.

Interpretation Approaches on NN

Approaches	Explains	Outcome	Type of Interpretation
Feature Visualization	Visualize what filters have learned	Shows patterns (edges, textures) that activate filters	Internal Representation
Activation Map	Visualize layer outputs for a given input	Highlights which parts of the image activate specific filters	Layer-wise Visualization
Saliency Map	Identify influential pixels	Gradient-based pixel importance for prediction	Gradient-based
Grad-CAM	Localize important regions for a class prediction	Heatmap over image showing class-specific focus areas	Gradient + Activation
LIME	Explain prediction via local surrogate model	Highlights image regions contributing to prediction	Perturbation-based
SHAP	Assign feature importance using game theory	Shapley values for pixel/region contribution	Model-agnostic, additive
Network Dissection	Match neurons to semantic concepts	Quantifies interpretability via concept alignment	Concept-based

CNN Interpretation

- Saliency maps, CAM (Class Activation Mapping), Grad-CAM
- Feature visualization to understand filter behavior

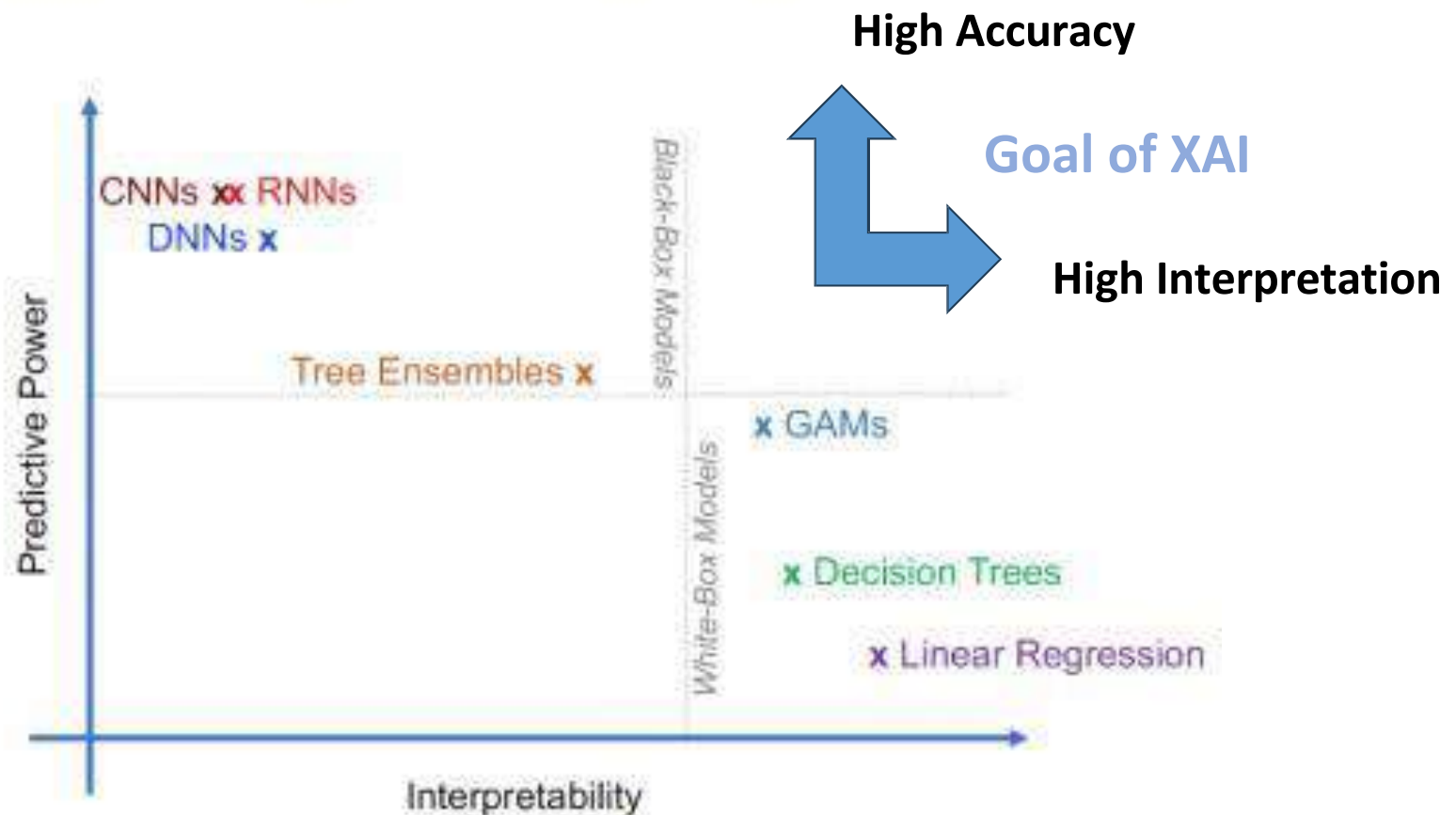
RNN and LSTM Interpretation

- Attention visualization
- Hidden state tracking
- Importance via sequence masking

Transformer Interpretation

- **Attention Maps:** Visualize which tokens attend to which
- **Attention Rollout** and **Layer-wise relevance propagation (LRP)**

Black box vs. White box models



Evaluation of Interpretability

Assesses how effectively a model or an explanation method helps users understand:

- **How the model works**
- **Why a particular prediction was made**
- **What factors influenced the decision**

Doshi-Velez and Kim (2017) propose three main levels for the evaluation of interpretability

- **Application level evaluation** (real task)
- **Human level evaluation** (simple task)
- **Function level evaluation** (proxy task)

Application-Grounded Evaluation

- Tests explanations in **real-world contexts** with domain experts.
- Measures **practical usefulness**—e.g., does it help doctors, bankers, etc., perform better?
- Most realistic but **resource-intensive**.

Human-Grounded Evaluation

- Uses **human judgment** in simplified settings (e.g., surveys, rating tasks).
- Focuses on **clarity, usability, and preference** of explanations.
- More scalable than application-grounded but less domain-specific.

Functionally-Grounded Evaluation

- Uses **automated, quantitative metrics** (no human involvement).
- Assesses explanation **fidelity, stability, and sensitivity**.
- Highly **scalable and objective**, but may not reflect human interpretability.

Evaluation - summary

Category	Who's Involved?	Example Tasks	Pros	Cons
Application-Grounded	Domain experts	Real-world decisions	High relevance & realism	Costly, slow, less scalable
Human-Grounded	Laypeople/crowdworkers	Simplified prediction tasks	Accessible, semi-realistic	Less context-specific
Functionally-Grounded	No humans	Fidelity, deletion/insertion	Fast, repeatable, objective	No insight into human use

Evaluation Type	Example	Focus
Application-Grounded	Real drivers using the app	Practical usefulness
Human-Grounded	Users rating clarity of routes	Human understanding
Functionally-Grounded	Algorithm tested via simulations	Internal correctness

References

- Molnar, Christoph. “Interpretable machine learning. A Guide for Making Black Box Models Explainable”, 2019.
<https://christophm.github.io/interpretable-ml-book/>.
- Explainable Artificial Intelligence: An Introduction to Interpretable Machine Learning, Uday Kamath: John Liu, Springer, ISBN 9783030833558
- <https://medium.com/dsaid-govtech/towards-a-comparable-metric-for-ai-model-interpretability-part-1-d55d4bae8a58>